Université d'Ottawa
Faculté de génie

École de science
d'informatique
et de génie électrique

uOttawa
L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

School of Electrical
Engineering
and Computer Science

Devoir 1
CSI2520 Paradigmes de Programmation
Hiver 2026

Assignment 1
CSI2120 Programming Paradigms
Winter 2026

8% - 12 points

Instructions :

- Les questions suivantes doivent être résolues en utilisant le langage de programmation Go
- Remettre vos réponses dans un fichier zip
- Ce devoir est individuel, le plagiat peut entrainer des conséquences importantes
- Ce devoir ne comptera à votre note finale que si vous obtenez 50% au cumul de vos examens de mi-session et final

Instructions :

- *You must answer the questions using the Go programming languages*
- *Submit your answers in a zip file*
- *This assignment is individual; plagiarism can lead to serious consequences.*
- *This assignment will only count toward your final grade if your combined midterm and final exam score is at least 50%*

Question 1. [4 points]

Le programme suivant calcule la somme des carrés d'une série de nombres en parallèle, mais il manque la synchronisation nécessaire pour garantir un résultat exact.
Utiliser `WaitGroup` et `Mutex` afin que ce programme fonctionne correctement.

The following program computes the sum of the squares of a series of numbers in parallel, but it lacks the synchronization mechanisms to ensure a correct result.
Use `WaitGroup` and `Mutex` to add the required synchronization.

```
package main

import (
     "fmt"
     "time"
)

// concurrently computes the square sum
// without synchronization?!
func parallelsquareSum(numbers []int) int {
     sum := 0
     for _, n := range numbers {
         go func(n int) {
          sq := n * n
          time.Sleep(500) // to simulate long compute time
          sum += sq
         }(n)
     }
     return sum
}

func main() {
     nums := []int{2, 3, 4, 5, 6, 8, 11, 15, 32, 77}
     fmt.Println("Total:", parallelsquareSum(nums))
}
```

Question 2. [4 points]

a) Ci-bas se trouve un module qui appelle de façon répétée une fonction. Celui-ci se termine lorsqu'un message au channel `stop` est envoyé. On voudrait ajouter une autre condition d'arrêt : le module devrait se terminer si son channel d'envoie n'est pas lu pour une période de 2 secondes. Indice : ceci peut se réaliser avec la fonction `time.After`.

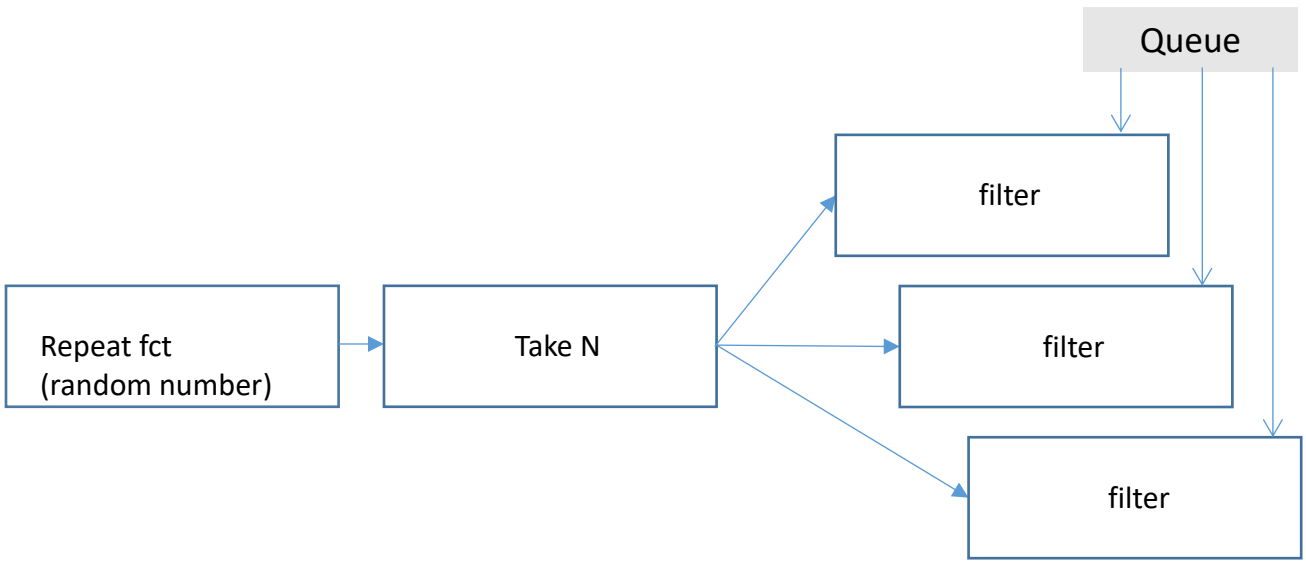
b) Avec ce module, créez de plus un pipeline composé de trois étapes concurrentes qui généreront des nombres de Harshad aléatoires. À la dernière étape, appliquez un fan-out de trois filtres Harshad (voici la figure de la page suivante). Au lieu d'envoyer les nombres de Harshad vers un canal de sortie, les trois filtres écrivent les nombres trouvés dans une même structure de données file (queue) qui leur est passée en paramètre (voir la signature de la fonction `filter` à la page suivante). Utilisez ce pipeline pour générer 200 nombres aléatoires; seuls ceux qui sont des nombres de Harshad sont insérés dans la file. Une fois le pipeline terminé, le contenu de la file est affiché par la fonction `main`.

```
func repeatFct(wg *sync.WaitGroup, stop <-chan bool, fct func() int)
<-chan int {

     intStream := make(chan int)

     go func() {
          defer func() {wg.Done()}()
          defer close(intStream)

          for {
             select {
                case <-stop:
                    fmt.Printf("\nFin de repeat ...\n")
                   return
                case intStream <- fct():
             }
          }
     }()

     return intStream
}
```

a) Above is a module that repeatedly calls a function. It stops when a message is sent to the `stop` channel. We want to add another stopping condition: the module should also terminate if its output channel is not read for a period of 2 seconds. Hint: this can be done using the `time.After` function.

b) *With the module in a), create a pipeline made of three concurrent stages that will generate random Harshad numbers. A fan-out of three Harshad filters is applied at the last stage. Instead of sending the Harshad numbers to an output channel, the three filters insert the found numbers to a common queue data structure passed to each filter (see the signature below). Use this pipeline to generate 200 random numbers; only the Harshad numbers are inserted into the queue. Once the pipeline terminates, the content of the queue is displayed by the* `main` *function.*

```
func filter(wg *sync.WaitGroup, stop <-chan bool,
            inputIntstream <-chan int,
            filter func(int) bool,
            outputQueue *Queue)
```



Question 3. [4 points]

La fonction ci-dessous produit des nombres premiers spéciaux se terminant par un suffixe donné. Un nombre maximum d'essais doit être aussi spécifié de façon à ce que la fonction puisse se terminer même lorsqu'aucun nombre premier spécial n'est trouvé.

The following function generates prime numbers that terminates by a given special pattern. A maximum number of trials is specified such that the function terminates even if the special prime cannot be found.

```
// a special prime is a prime number that ends
// with the specified pattern sequence
// after nTrials the function returns with a false error code
func getSpecialPrime(pattern int64, maxValue int64, nTrials int)
                                                    (int64, bool) {

   var div int64
   for div=10; pattern/div!=0; div*=10 {

   }

   for i:=0; i<nTrials; i++ {
      n:= getPrime(maxValue)
      if n%div==pattern {
         return n, true  // special prime found
      }
   }

   return 0, false // we failed to find a special prime
}
```

```
// checks if it is a prime number
func isPrime(v int64) bool {

   sq:= int64(math.Sqrt(float64(v)))+1
   var i int64
   for i=2; i<sq; i++ {

      if v%i == 0 {
         return false
      }
   }
   return true
}



// returns a prime number
func getPrime(maxValue int64) int64 {

   for {
      n:= rand.Int63n(maxValue)
      if isPrime(n) {
         return n
      }
   }
}
```

Le programme ci-dessous génère 4 nombres premiers spéciaux se terminant par le suffixe 1111.Ce programme n'est pas très efficace puisqu'il prend plusieurs secondes à s'exécuter (à noter aussi comment le programme s'arrêtera automatiquement lorsque la recherche des nombres spéciaux excède 10 minutes).

The program below generates 4 special primes ending with the suffix 1111. This program is not very efficient; it takes several seconds to run (note also how the program will stop if the special number generation takes more than 10 minutes).

```
// This is the version without threads
func main() {

   fmt.Println("Solution without threads.")

   var number int64
   var sp []int64

   var pattern int64= 1111 // the suffix pattern we are looking for
   var maxV int64= 1000000000000   // maximum value for the prime
   var trials int= 100   // number of trials for each attempts
   var nPrimes int=4     // number of special primes we want

   start:= time.Now()

   for len(sp)<nPrimes {
     number,_= getSpecialPrime(pattern,maxV,trials)
      if number>0 { // we have found a special prime
          sp= append(sp, number)
      }

      // give up after 10 minutes
      current:= time.Now()
      if current.Sub(start) > time.Minute*10 {
          break
      }
   }

   end:= time.Now()
   fmt.Println("Special prime numbers are: ",sp)
   fmt.Println("End of program.",end.Sub(start))
}
```

Solution without threads.
Special prime numbers are: [549559081111 975098461111 491181341111 359545511111]
End of program. 17.8778324s

Afin d'améliorer la vitesse de calcul de ce programme, nous vous demandons de créer plusieurs fils chargés de rechercher ces 4 nombres premiers spéciaux. Pour ce faire, il suffit d'appeler plusieurs fois une goroutine qui crée un thread de recherche de nombres premiers spéciaux et qui écrit les résultats dans un channel donné (tous les fils doivent écrire dans le même channel). L'extrait de code ci-dessous constitue un point de départ pour écrire votre fonction `main`.

Nous vous demandons de tester votre programme avec 1, 2, 4, 8, 16 et 32 fils. Présentez les résultats dans un tableau (en moyennant les résultats de chaque cas sur 10 essais). N'oubliez pas d'inclure, dans votre remise, le fichier PDF contenant ce tableau.

To improve the computational speed of this program, we ask you to create several threads searching for these 4 special prime numbers. This can be done by simply calling several times a goroutine creating a special prime number searching thread that writes the results to a specified channels (we want all threads to write to the same channel). The code snippet below is a starting point that you can use to write your `main` *function.*

We ask you to test your program with 1, 2, 4, 8, 16 and 32 threads. Provide the results in a table (average the results for each case over 10 trials). Do not forget to include the pdf file showing this table in your submission.

```
stop:= make(chan bool)
result:= make(chan int64)
defer close(result)

var wg sync.WaitGroup

nThreads:= 8

wg.Add(nThreads)
for i:=0;i<nThreads;i++ {
  go createSpecialPrimeStream(result,&wg,stop,pattern,maxV,trials)
}
```

```
Solution with all threads writing to the same channel.
Number of threads: 8
Special prime numbers are:  [634819211111 827518991111 381310681111
502732141111]
End of program. 1.3390992s
```